

Hackathon-Ready Project Document: Taskmint (TNImpact)

What it is

Taskmint is a lightweight **micro-business operating system** that lets a small seller:

- create a **public storefront**,
 - manage **products + inventory**,
 - run the **order lifecycle** (accept → ship → complete),
 - generate **tax invoices (print-ready)**,
 - send **transactional emails** (status updates) and **bulk announcements**,
 - track **sales analytics + downloadable reports**,
 - build **custom database tables** visually (“Database Builder”) and store structured business data.
-

Core modules (everything included)

Frontend modules (Vite + React)

Location: d:\TNImpact\frontend\src\

Routing / Pages (full list)

- **Auth**
- LoginPage.jsx (/login)
- **Seller (authenticated)**
- HomePage.jsx (/)
- DashboardPage.jsx (/dashboard)
- ProductsPage.jsx (/products)
- ActiveOrdersPage.jsx (/orders)
- DataPage.jsx (/data/:formId)
- VisualBuilderPage.jsx (/builder) → **Database Builder**
- WorkflowPage.jsx (/workflows) → **Automations**
- AnnouncementsPage.jsx (/announcements)
- **Customer (public)**

- CustomerStorePage.jsx (/store?user=<seller_email>)

Shared UI / State

- **Theme**
- context/ThemeContext.jsx
- Uses data-theme="light|dark" + CSS variables in src/index.css
- Persists to localStorage key taskmint_theme
- **Toasts (in-app notifications, no browser alerts)**
- context/ToastContext.jsx
- Bottom-right toast messages with auto-dismiss

Key component

- **Invoice modal**
- components/InvoicePrint.jsx
- Shows a **portrait tax invoice** and supports **Print** (browser print dialog) using an off-screen iframe.

Styling

- src/index.css implements:
- glass panels/cards/inputs
- dark/light themes
- consistent primary button styling via .btn-primary
- dropdown styling overrides

Frontend dependencies used

- **Core:** react, react-dom, react-router-dom
 - **Auth:** @react-oauth/google, jwt-decode
 - **UI:** tailwindcss (v4), lucide-react
 - **Charts:** chart.js, react-chartjs-2
 - **Drag & drop:** react-dnd, react-dnd-html5-backend
 - **PDF capability present:** jsPDF is installed (invoice PDF generator code exists in InvoicePrint.jsx, UI button currently removed per your instruction)
-

Backend modules (Flask + SQLAlchemy + SQLite)

Location: d:\TNImpact\backend\

Entry / wiring

- app.py
- Flask app, enables CORS
- SQLite DB file: backend/msme_builder.db
- registers routes.bp under /api
- routes.py
- all API endpoints + business logic
- models.py
- SQLAlchemy models (tables)
- DB: SQLite, auto-created with db.create_all() on startup

Email module (Gmail SMTP)

- Implemented in routes.py using smtplib.SMTP_SSL('smtp.gmail.com', 465)
- Uses:
- **sender** = seller login email (User.email)
- **password** = seller app password stored in User.sender_password (or env fallback)

Automations module

- Stored rules in automations table
- Runtime automation check: low stock → returns a WhatsApp deep link (does not send WhatsApp directly)

Data model (everything stored)

From backend/models.py:

Commerce

- **User:** email, name(company_name), password, sender_password, (also sender_email, logo_url fields exist)
- **Product:** name, price, stock, gst_slab, gst_type, etc.

- **Order:** order_number, status, customer details, totals, GST totals, notes (if provided), timestamps
- **OrderItem:** product snapshot + qty + unit price + GST info + subtotal
- **OrderTracking:** status timeline entries
- **SalesRecord:** created on completion

Dynamic database tables (“Forms system”)

- **Form** (table definition)
- **FormField** (schema columns)
- **Record** (row)
- **RecordValue** (cell values)

Automation rules

- **Automation:** triggers/conditions/actions + config JSON

Feature set (complete list)

A) Authentication & onboarding

- **Sign up / login** with email & password
- **Google sign-in** support (frontend Google OAuth integration)
- Stores logged-in user in localStorage key taskmint_user
- Optional **Gmail App Password** entry for sending emails:
- Stored once in DB (users.sender_password)
- Used automatically for all future transactional/bulk emails

B) Public customer storefront

- Public URL format:
- Store page: /store?user=<seller_email>
- Features:
- Welcome banner: “Welcome to <company>” (via GET /api/store/info)
- Product listing (only stock > 0)

- Fuzzy-like search (substring match) on product name
 - Cart
 - Checkout form collecting customer details
 - Places order into seller's order queue
-

C) Product & inventory management (seller)

- Add / edit / delete products
 - GST selection:
 - Predefined slabs (5/18/40/0)
 - Manual entry (CGST/SGST/IGST) stored as an effective combined slab value
 - Inventory stock is decremented when an order is **completed** (delivery confirmed)
-

D) Order management workflow (seller)

- Views and manages orders by status:
- **Pending** (accept/reject)
- **Accepted** (ship)
- **Shipped** (complete)
- **Completed**
- Confirmations:
- Accept / Reject confirmations in-app
- "Clear orders" confirmation in-app
- Search:
- Searches by order number, customer name/email, and product names (substring matching)
- Actions:
- Accept → sends customer email
- Reject → sends customer email (includes refund message)
- Ship → sends customer email

- Complete → sends customer email, writes sales record, decrements stock
-

E) Invoice generation (portrait print-ready)

- Invoice modal shows:
 - seller name/logo (logo optional)
 - order number & date
 - Bill To details
 - itemized list with prices & GST
 - totals area
 - Print:
 - Uses a hidden iframe + `@page { size: A4 portrait; }`
 - Opens the browser print dialog
-

F) Email notifications (transactional)

Order status emails include:

- order metadata
- customer details
- full item list with GST amounts
- totals + refund note on rejection

Email transport:

- Gmail SMTP SSL
 - sender: seller login email
 - password: stored Gmail app password
-

G) Announcements (bulk broadcast)

- Seller can send an email announcement to:
- all distinct customer emails who have placed an order
- Supports scheduling UI (implemented client-side using `setTimeout`)

H) Analytics + downloadable sales reports

- Dashboard:
- sales chart (line)
- order status distribution (doughnut)
- KPIs
- Downloadable CSV sales report for month/year:
- one row per completed order item
- includes GST and totals

I) Database Builder (visual schema builder)

- Drag & drop “table builder”
- Field types:
- Text, Number, Date, Checkbox, File Upload, Image
- Save structure → creates a **Form** with fields in backend
- Data page auto-generates a grid UI from schema and supports adding records

J) Default “Orders” database table auto-created & populated

- When an order is created, backend ensures a default **Form** named Orders exists.
- It adds a record capturing key order details (order number, customer, product summary, total, status, created timestamp).

K) Automations (rule builder + execution)

- Create “IF / THEN” automation rules in UI
- Backend checks:
- inventory low stock rules
- At order completion:
- may return a WhatsApp deep link for a low-stock alert

- UI shows a prompt to open WhatsApp

Architecture overview (how it works)

Browse GET GET POST GET/POST/PUT/DELETE GET/POST POST GET POST POST POST GET/POST
PrintCustomerBrowserStoreUIStoreInfoAPIStoreProductsAPIStoreOrdersAPISellerUIProductsAPIOrdersAPIOrderStatusAPIsReportsAPIAutomationsAPIAnnouncementsAPIFormsAPIRecordsAPISQLiteDBGmailSMTPInvoiceIframe

Complete API surface (backend endpoints)

All are prefixed with /api.

Auth

- POST /auth/signup
- POST /auth/login
- POST /auth/google

Products

- GET /products
- POST /products
- GET /products/<id>
- PUT /products/<id>
- DELETE /products/<id>

Storefront (public)

- GET /store/info
- GET /store/products
- POST /store/orders
- GET /store/orders/<order_number>/track (track UI removed from customer page, but endpoint exists)

Orders (seller)

- GET /orders
- POST /orders

- DELETE /orders/clear
- POST /orders/<id>/accept
- POST /orders/<id>/reject
- POST /orders/<id>/ship
- POST /orders/<id>/complete
- GET /orders/<id>/track
- GET /orders/<id>/invoice

Reports

- GET /reports/sales
- GET /reports/sales/download

Automations

- GET /automations
- POST /automations
- PUT /automations/<id>
- DELETE /automations/<id>
- POST /automations/check

Announcements

- POST /announcements/send

Forms (Database Builder)

- GET /forms
- POST /forms
- GET /forms/<id>
- DELETE /forms/<id>
- GET /forms/<id>/records
- POST /forms/<id>/records

User settings

- GET /users/settings
- PUT /users/settings

Demo script (what to show judges)

- **Step 1:** Login (email/password or Google)
- **Step 2:** Add a product (with GST)
- **Step 3:** Copy store link and open it in an incognito window
- **Step 4:** Customer places an order
- **Step 5:** Seller accepts → ships → completes the order (show status emails)
- **Step 6:** Open invoice and print (portrait)
- **Step 7:** Dashboard charts and download CSV report
- **Step 8:** Database Builder: create a table, add records
- **Step 9:** Announcements: send a broadcast to prior customers
- **Step 10:** Automations: set low stock rule, complete an order and show WhatsApp link prompt

Judge questions you will get (with strong answers)

Product & value

- **Q: What problem does this solve?**

A: It consolidates storefront, order operations, invoicing, and customer communications into one lightweight tool for small sellers—reducing manual work and providing a repeatable workflow.

- **Q: What's unique compared to generic e-commerce tools?**

A: The “Database Builder” is a differentiator: sellers can create custom tables for their own business needs (customers, suppliers, tasks, etc.) without coding, alongside commerce features.

Technical architecture

- **Q: What is your stack and why?**

A: React + Vite for fast UI iteration, Flask for a simple API, SQLite for hackathon-friendly persistence, and modular features (email, reports, automations) built on top.

- **Q: How do you handle printing invoices?**

A: Client-side print via a dedicated invoice template rendered into a hidden iframe with A4 portrait print CSS, so it prints consistently without server-side PDF infrastructure.

Data & modules

- **Q: How do you model orders and items?**

A: Orders are stored with customer info, totals, timestamps; items store product snapshot and tax fields. OrderTracking captures lifecycle events.

- **Q: How do the dynamic tables work?**

A: Forms define schema (fields); Records store rows; RecordValues store values per field—allowing flexible tables built visually.

Emails & communications

- **Q: How do emails work and what's required?**

A: We use Gmail SMTP with an App Password stored once for the seller. Emails are sent on accept/reject/ship/complete and announcements reuse the same sender credential.

- **Q: What about deliverability and reliability?**

A: For hackathon demo, Gmail SMTP is straightforward. For production we'd move to a transactional provider (SendGrid/Mailgun), add retries/queues, and show delivery status.

Automations

- **Q: What “automations” exist right now?**

A: Inventory low-stock detection that returns a WhatsApp deep link so the seller can notify a manager immediately. The UI supports additional rule definitions.

Security & production readiness (common judge angle)

- **Q: Is authentication production-ready?**

A: Not yet—this is hackathon-mode. Next steps include proper auth tokens, password hashing, role checks, and authorization enforcement per resource.

- **Q: What's your scaling plan?**

A: Replace SQLite with Postgres, add background job workers for email/announcements, and enforce inventory reservation to prevent overselling under concurrency.

Metrics & impact

- **Q: How would you measure success?**

A: Reduced time to process orders,
improved repeat purchases through announcements,
fewer errors in invoicing/reporting, and adoption of custom tables by SMEs.

Limitations (what to say confidently)

- **Scheduling announcements** is currently browser-based (for demo).
Production would use a backend scheduler/queue.
- **Gmail SMTP** is fine for demo; production needs a proper email service + observability.
- **Auth/security** is simplified for hackathon speed.